

Reporte Técnico - Proyecto Final EDA

Construcción de la estructura:

Aquí he creado 3 carpetas independientes (baseline propuesto en mi exposición)

```
venv ~/San Pablo/Semestre_26-13/Estructura de Datos Avanzados git:(main) 1936 · +8686002 -348 (0.032s)
tree -L 1 ProyectoFinal_EDA
ProyectoFinal_EDA
├── __pycache__
├── backup.py
├── chroma
├── chroma-hnswlib
├── dataset.py
├── experimentos.py
└── venv
```

```
venv ~/San Pablo/Semestre_26-13/Estructura de Datos Avanzados git:(main) 1935 · +8685952 -348 (0.032s)
tree -L 1 baseline_mnru_portado
baseline_mnru_portado
├── chroma
├── chroma-hnswlib
├── dataset.py
├── experimentos.py
└── venv
```

```
venv ~/San Pablo/Semestre_26-13/Estructura de Datos Avanzados git:(main) 1935 · +8685952 -348 (0.033s)
tree -L 1 baseline_hnsw_ru
baseline_hnsw_ru
├── chroma
├── chroma-hnswlib
├── dataset.py
├── experimentos.py
└── venv
```

Diferencias entre los 3:

	Base	MN-RU paper	MN-RU adaptativo
Reconexión mutua	No	Si	Si
Backup index	No	Si	Si
Dual search	No	Si	Si
Detección de inalcanzables	No	K-NN $O(N^2)$	Contador $O(1)$
Tau	No	40k fijo	Adaptativo
Persistencia del backup	No	No	Si

Repositorio de GitHub para descargar el código y compilar:

https://github.com/FabrizioMessa/TRABAJO_FINAL_EDA.git

Archivos modificados:

Archivo 1: chroma-hnswlib/hnswlib/hnswalg.h

```
// ===== MI APORTE =====
std::vector<int> in_degree_;
std::unordered_set<tableint> unreachable_set_;
mutable HierarchicalNSW<dist_t> *backup_index_ = nullptr;
mutable size_t backup_trigger_threshold_ = 40000;
mutable size_t operation_counter_ = 0;
SpaceInterface<dist_t> *space_ = nullptr;

mutable size_t tau_base_ = 40000;
mutable double alpha_ = 10.0;
mutable double beta_ = 5.0;
mutable size_t recalc_interval_ = 1000;
mutable size_t ops_since_recalc_ = 0;
mutable double baseline_recall_ = 1.0;
// ===== MI APORTE =====
```

dentro de la función HierarchicalNSW()

```
// ===== MI APORTE =====
space_ = s;
// =====
```

```
// ===== MI APORTE =====
in_degree_.assign(max_elements_, 0);
space_ = s;
// =====
```

dentro del destructor ~HierarchicalNSW()

```
// ===== MI APORTE =====
if (backup_index_ != nullptr) {
    delete backup_index_;
    backup_index_ = nullptr;
}
// =====
```

dentro de la función: mutuallyConnectNewElement()

```
// ===== MI APORTE =====
in_degree_[selectedNeighbors[idx]]++;
// ===== MI APORTE =====
```

```
// ===== MI APORTE =====
in_degree_[cur_c]++;
// ===== MI APORTE =====
```

```
// ===== MI APORTE =====
std::unordered_set<tableint> old_neighbors(data, data + sz_link_list_other);
// ===== MI APORTE =====
```

```
// ===== MI APORTE =====
std::unordered_set<tableint> new_neighbors(data, data + indx);
for (auto old_neigh : old_neighbors) {
    if (new_neighbors.find(old_neigh) == new_neighbors.end()) {
        in_degree_[old_neigh]--;
        // si llega a 0 es inalcanzable
        if (in_degree_[old_neigh] == 0) {
            unreachable_set_.insert(old_neigh);
        }
    }
}

for (auto new_neigh : new_neighbors) {
    if (old_neighbors.find(new_neigh) == old_neighbors.end()) {
        in_degree_[new_neigh]++;
        unreachable_set_.erase(new_neigh);
    }
}
// ===== MI APORTE =====
```

dentro de la función: resizeIndex()

```
// ===== MI APORTE =====
in_degree_.resize(new_max_elements, 0);
// =====
```

dentro de la función saveIndex()

```
std::string getBackupDir() {
    if (persist_on_write_) return persist_location_;
    size_t sep = persist_location_.find_last_of("/\\");
    if (sep != std::string::npos) return persist_location_.substr(0, sep);
    return ".";
}
```

creo la nueva función saveBackupIndex()

```
void saveBackupIndex() {
    if (backup_index_ == nullptr && unreachable_set_.empty())
        return;

    std::string dir = getBackupDir();
    std::string archivo_meta = dir + "/backup.bin";

    std::ofstream output(archivo_meta, std::ios::binary);
    if (!output.is_open()) {
        throw std::runtime_error("No se pudo abrir backup.bin para escritura");
    }

    size_t tam = unreachable_set_.size();
    writeBinaryPOD(output, tam);
    for (auto id : unreachable_set_) {
        writeBinaryPOD(output, id);
    }

    writeBinaryPOD(output, operation_counter_);
    writeBinaryPOD(output, backup_trigger_threshold_);
    output.close();

    if (backup_index_ != nullptr && backup_index_>getCurrentElementCount() > 0) {
        std::string archivo_indice = dir + "/backup_index.bin";
        backup_index_>saveIndex(archivo_indice);
    }
}
```

creo la nueva función loadBackupIndex()

```
void loadBackupIndex() {
    std::string dir = getBackupDir();
    std::string archivo_meta = dir + "/backup.bin";

    std::ifstream input(archivo_meta, std::ios::binary);
    if (!input.is_open())
        return;

    size_t tam;
    readBinaryPOD(input, tam);
    unreachable_set_.clear();
    for (size_t i = 0; i < tam; i++) {
        tableint id;
        readBinaryPOD(input, id);
        unreachable_set_.insert(id);
    }

    readBinaryPOD(input, operation_counter_);
    readBinaryPOD(input, backup_trigger_threshold_);
    input.close();

    std::string archivo_indice = dir + "/backup_index.bin";
    std::ifstream test(archivo_indice, std::ios::binary);
    if (test.is_open()) {
        test.close();
        if (backup_index_ != nullptr)
            delete backup_index_;
        backup_index_ = new HierarchicalLNSW<dist_t>(space_, archivo_indice, false);
    }
}
```

dentro de la función persistDirty()

```
// ===== MI APORTE =====
if (!unreachable_set_.empty()) {
    saveBackupIndex();
}
// ===== MI APORTE =====
```

dentro de la función loadPersistedIndex()

```
// ===== MI APORTE =====
space_ = s;
// =====
```

```
// ===== MI APORTE =====
in_degree_.assign(max_elements_, 0);
if (persist_on_write_) {
    loadBackupIndex();
}
// ===== MI APORTE =====
```

dentro de la función loadIndex()

```
// ===== MI APORTE =====
space_ = s;
// =====
```

```
// ===== MI APORTE =====
in_degree_.assign(max_elements_, 0);
if (persist_on_write_) {
    loadBackupIndex();
}
// ===== MI APORTE =====
```

dentro de la función markDeletedInternal()

```
// ===== MI APOORTE =====
operation_counter_++;
ops_since_recalc_++;

if (ops_since_recalc_ >= recalc_interval_) {
    recalcTau();
    ops_since_recalc_ = 0;
}

if (in_degree_[internalId] == 0) {
    unreachable_set_.insert(internalId);
}

// ===== MI APOORTE =====
```

creo las nuevas funciones rebuildBackupIndex() y recalcTau()

```
// ===== MI APOORTE =====
void rebuildBackupIndex() {
    if (backup_index_ != nullptr) {
        delete backup_index_;
    }

    backup_index_ = new HierarchicalNSW<dist_t>(space_, max_elements_, M_, ef_construction_);

    for (auto id : unreachable_set_) {
        if (!isMarkedDeleted(id)) {
            continue;
        }
        labeltype label = getExternalLabel(id);
        backup_index_>addPoint(getDataByInternalId(id), label);
    }
}

void recalcTau() {
    double frac_inalc = 0.0;
    if (cur_element_count > 0) {
        frac_inalc = (double)unreachable_set_.size() / (double)cur_element_count;
    }
    double caida_recall = 1.0 - baseline_recall_;
    double nuevo_tau = (double)tau_base_ * (1.0 + alpha_ * frac_inalc + beta_ * caida_recall);
    backup_trigger_threshold_ = (size_t)std::min(nuevo_tau, (double)tau_base_ * 5.0);
}

// ===== MI APOORTE =====
```

dentro de la función addPoint()

```
// ===== MI APOORTE =====
unreachable_set_.erase(internal_id_replaced);
// =====
```

dentro de la función updatePoint()

```
// ===== MI APOORTE =====
std::vector<tableint> listTwoHop = getConnectionsWithLock(elOneHop, layer);
for (auto &&elTwoHop : listTwoHop) {
    std::vector<tableint> twoHopNeighbors = getConnectionsWithLock(elTwoHop, layer);
    bool is_mutual = false;
    for (auto &&cn : twoHopNeighbors) {
        if (cn == internalId) {
            is_mutual = true;
            break;
        }
    }

    if (is_mutual) {
        sCand.insert(elTwoHop);
    }
}

// ===== MI APOORTE =====
```

dentro de la función searchknn()

```
// ===== MI APORTE =====
if (operation_counter_ >= backup_trigger_threshold_ && !unreachable_set_.empty()) {
    const_cast<HierarchicalNSW<dist_t> *>(this)->rebuildBackupIndex();
    operation_counter_ = 0;
}

if (backup_index_ != nullptr && backup_index_->getCurrentElementCount() > 0) {
    auto backup_result = backup_index_->searchKnn(query_data, k, isIdAllowed);
    while (backup_result.size() > 0) {
        auto rez = backup_result.top();
        backup_result.pop();

        if (result.size() < k) {
            result.push(rez);
        } else if (rez.first < result.top().first) {
            result.pop();
            result.push(rez);
        }
    }

    while (result.size() > k) {
        result.pop();
    }
}

// ===== MI APORTE =====
```

Archivo 2: chroma-hnswlib/python_bindings/bindings.cpp

```
// ===== MI APORTE =====
size_t getUnreachableCount() const {
    return appr_alg->unreachable_set_.size();
}

bool hasBackupIndex() const {
    return appr_alg->backup_index_ != nullptr;
}

size_t getBackupCount() const {
    if (appr_alg->backup_index_ != nullptr) {
        return appr_alg->backup_index_->getCurrentElementCount();
    }
    return 0;
}

size_t getOperationCounter() const {
    return appr_alg->operation_counter_;
}

size_t getTau() const {
    return appr_alg->backup_trigger_threshold_;
}

void setBaselineRecall(double recall) {
    appr_alg->baseline_recall_ = recall;
}

void setTau(size_t tau) {
    appr_alg->backup_trigger_threshold_ = tau;
}

void setTauParams(double alpha, double beta, size_t interval) {
    appr_alg->alpha_ = alpha;
    appr_alg->beta_ = beta;
    appr_alg->recalc_interval_ = interval;
}

void rebuildBackup() {
    appr_alg->rebuildBackupIndex();
}

std::vector<int> getInDegreeList() const {
    return appr_alg->in_degree_;
}

// ===== MI APORTE =====
```

```
// ===== MI APORTE =====
.def("get_unreachable_count", &Index<float>::getUnreachableCount)
.def("has_backup_index", &Index<float>::hasBackupIndex)
.def("get_backup_count", &Index<float>::getBackupCount)
.def("get_operation_counter", &Index<float>::getOperationCounter)
.def("get_tau", &Index<float>::getTau)
.def("set_baseline_recall", &Index<float>::setBaselineRecall, py::arg("recall"))
.def("set_tau", &Index<float>::setTau, py::arg("tau"))
.def("set_tau_params", &Index<float>::setTauParams, py::arg("alpha"), py::arg("beta"), py::arg("interval"))
.def("rebuild_backup", &Index<float>::rebuildBackup)
.def("save_backup", [](Index<float> &idx)
    { idx.appr_alg->saveBackupIndex(); })
.def("load_backup", [](Index<float> &idx)
    { idx.appr_alg->loadBackupIndex(); })
.def("get_in_degree_list", &Index<float>::getInDegreeList)
// ===== MI APORTE =====
```

Archivo 3: chroma/chromadb/segment/impl/vector/hnsw_params.py

```
# ===== MI APORTE =====
"hnsw:mru_enabled": lambda p: isinstance(p, bool),
"hnsw:mru_tombstone_threshold": lambda p: isinstance(p, int),
# ===== MI APORTE =====
```

```
# ===== MI APORTE =====
mru_enabled: bool
mru_tombstone_threshold: int
# ===== MI APORTE =====
```

```
# ===== MI APORTE =====
self.mru_enabled = bool(metadata.get("hnsw:mru_enabled", True))
self.mru_tombstone_threshold = int(metadata.get("hnsw:mru_tombstone_threshold", 100))
# ===== MI APORTE =====
```

Archivo 4: chroma/chromadb/segment/impl/vector/local_hnsw.py

```
# ===== MI APORTE =====
if self._mru_params.get("enabled", False):
    tombstone_count = index.get_unreachable_count()
    if tombstone_count >= self._mru_params.get("tombstone_threshold", 100):
        index.rebuild_backup()
# ===== MI APORTE =====
```

Archivo 5: chroma/chromadb/segment/impl/vector/local_persistent_hnsw.py

```
# ===== MI APORTE =====
self._mru_params = {
    "enabled": self._params.mru_enabled,
    "tombstone_threshold": self._params.mru_tombstone_threshold,
}
if self._params.mru_enabled:
    index.load_backup()
# ===== MI APORTE =====
```


Evaluación de Rendimiento

Dataset: 10k de vectores y con dimensión 100

Escenario de verificación:

- 100 ciclos delete+insert (50 vectores c/u) = 5,000 operaciones.
- $M = 16$, $ef_construction = 200$, $ef = 100$.
- Self-query recall: se insertan 200 vectores nuevos al final y se verifica que cada uno se encuentre a si mismo como top 1.

Tres baselines comparados:

- HNSW Stock: ChromaDB base.
- MN-RU Paper: Algoritmo Xiao et al. (K-NN $O(N^2)$, $\tau=40000$ fijo)
- MN-RU Adaptativo: Mi propuesta (contador $O(1)$, τ adaptativo,

```
python3 dataset.py
MN-RU ADAPTATIVO
Seed 42: Recall=1.0000 Inalc=71 Tau=42800 Bkp=0 Lat=0.0573ms
Seed 123: Recall=0.9950 Inalc=93 Tau=43400 Bkp=0 Lat=0.0563ms
Seed 456: Recall=1.0000 Inalc=78 Tau=42960 Bkp=0 Lat=0.0550ms
```

```
python3 dataset.py
HNSW BASE
Seed 42: Recall=1.0000 (200/200) Lat=0.0570ms
Seed 123: Recall=1.0000 (200/200) Lat=0.0556ms
Seed 456: Recall=0.9900 (198/200) Lat=0.0561ms
```

```
python3 dataset.py
MN-RU PAPER
Seed 42: Recall=1.0000 Inalc=100 Ops=5000 Lat=0.0564ms
Seed 123: Recall=0.9950 Inalc=96 Ops=5000 Lat=0.0567ms
Seed 456: Recall=1.0000 Inalc=80 Ops=5000 Lat=0.0557ms
```

```
python3 experimentos.py
=====
EXPERIMENTOS MN-RU ADAPTATIVO
Dataset: 10000 vectores, dimension = 100
=====
```

Full Coverage				
Metrica	Baseline		MN-RU	p-valor
Recall	0.9980 +/- 0.0040	1.0000 +/- 0.0000	1.0000	
Latencia (ms)	0.0559 +/- 0.0005	0.0563 +/- 0.0008		
Inalcanzables	125.6000 +/- 22.5973	124.8000 +/- 11.5482		
Backup count	0.0000 +/- 0.0000	0.0000 +/- 0.0000		

Random				
Metrica	Baseline		MN-RU	p-valor
Recall	0.9940 +/- 0.0080	0.9960 +/- 0.0049	1.0000	
Latencia (ms)	0.0542 +/- 0.0011	0.0540 +/- 0.0013		
Inalcanzables	44.0000 +/- 8.8544	44.2000 +/- 10.5338		
Backup count	0.0000 +/- 0.0000	0.0000 +/- 0.0000		

New Data				
Metrica	Baseline		MN-RU	p-valor
Recall	0.9980 +/- 0.0040	1.0000 +/- 0.0000	1.0000	
Latencia (ms)	0.0606 +/- 0.0007	0.0608 +/- 0.0006		
Inalcanzables	184.6000 +/- 15.8063	172.0000 +/- 45.3872		
Backup count	0.0000 +/- 0.0000	0.0000 +/- 0.0000		